

UNIVERSIDAD DE LA COSTA (CUC)

FACULTAD DE INGENIERÍA

PROGRAMA DE INGENIERÍA DE SISTEMAS

SÉPTIMO SEMESTRE

ENTREGABLES PROYECTO — SEGUNDO CORTE (20%)

API REST PARA LA GESTIÓN DE UNA BIBLIOTECA PERSONAL

Catálogo de libros con seguimiento de lectura, calificaciones y estadísticas

PRESENTADO POR:

ISAÍAS JOSÉ SÁNCHEZ CERVANTES

PRESENTADO A:

ING. LUIS TOSCANO CASTILLA

Docente — Asignatura de Desarrollo Web

BARRANQUILLA, ATLÁNTICO

MAYO DE 2026

Tabla de contenido

1. Contexto
2. Objetivos
3. Justificación
4. Alcance del proyecto
5. Herramientas
6. Repositorio de código

1. Contexto

1.1 Explicación breve

El presente proyecto consiste en el diseño e implementación de una API REST para la gestión de una biblioteca personal de libros. El sistema permite registrar, consultar, actualizar y eliminar libros de una colección, llevar el seguimiento del estado de lectura de cada uno (pendiente, leyendo o leído), asignar calificaciones del 1 al 5 y escribir notas personales. Adicionalmente, expone un endpoint de estadísticas que permite obtener un resumen del catálogo.

El proyecto incorpora un frontend completo con interfaz visual de estantería de biblioteca, desarrollado con HTML, CSS y JavaScript vanilla, accesible desde la misma URL del servidor.

La aplicación se construyó con Node.js y Express, una de las tecnologías permitidas por la asignatura, aprovechando su ecosistema maduro de middlewares de seguridad y validación.

1.2 Problema que resuelve

Los lectores habituales acumulan decenas o cientos de libros sin un sistema organizado que les permita recordar qué han leído, qué están leyendo, qué quieren leer y qué tan bueno les pareció cada libro. Las soluciones existentes son aplicaciones de terceros que no permiten personalización ni integración con otros sistemas.

Este proyecto plantea cómo construir una solución propia y extensible siguiendo las buenas prácticas de desarrollo de APIs REST: separación entre la capa de validación de entradas, la capa de lógica de negocio y la capa de acceso a datos, manejo de errores HTTP estándar, logging estructurado y pruebas automatizadas.

1.3 Solución propuesta

Se propone construir una API REST con los siguientes componentes:

- Un modelo de libro con atributos descriptivos (título, autor, género, año, páginas) y de seguimiento (estado de lectura, calificación, notas personales).
- Siete operaciones que permitan crear, listar con filtros, buscar de forma tolerante a acentos, consultar por ID, actualizar, cambiar estado rápidamente y eliminar libros.
- Un endpoint de estadísticas que retorne métricas agregadas del catálogo (total por estado, por género, calificación promedio, mejores calificados).
- Validación de todos los inputs con Joi antes de que lleguen a la lógica de negocio.
- Logging estructurado con Pino para registrar cada petición HTTP y cada operación de escritura.
- Documentación interactiva generada con Swagger UI y un archivo OpenAPI 3.0.
- Pruebas automatizadas con Jest y Supertest que cubran todos los endpoints.
- Frontend visual de estantería con panel deslizante de detalle, cambio rápido de estado y calificación, y búsqueda en tiempo real.

1.4 Resultados esperados

Al finalizar el proyecto se espera entregar:

- Una API REST funcional, accesible públicamente en producción.
- Documentación interactiva en `/api-docs` (Swagger UI).
- Cumplimiento del 100% de los requerimientos de la asignatura.
- Suite de pruebas automatizadas con cobertura de los endpoints expuestos.
- Repositorio público en GitHub con el código fuente, las pruebas y la documentación.
- Frontend accesible desde la URL raíz del proyecto desplegado.

2. Objetivos

2.1 Objetivo general

Desarrollar una API REST completa con frontend integrado que permita a cualquier usuario gestionar su colección personal de libros, aplicando los principios de diseño de APIs REST, validación de datos, separación de capas, manejo de errores HTTP estándar, pruebas automatizadas y despliegue en la nube.

2.2 Objetivos específicos

- Diseñar un modelo de datos para representar libros con atributos descriptivos y de seguimiento de lectura, validados con restricciones en la base de datos y en la capa de esquemas.
- Implementar el conjunto completo de operaciones CRUD necesarias para la gestión de libros, incluyendo filtrado por estado y género, búsqueda tolerante a acentos y paginación.
- Desarrollar un endpoint de estadísticas que agregue métricas del catálogo en una sola consulta.
- Separar la lógica de negocio de los controladores mediante una capa de servicios (`services.js`) y una capa de validación (`schemas.js`).
- Aplicar middlewares de seguridad (Helmet) y control de uso (rate limiting) para proteger la API en producción.
- Construir una suite de pruebas automatizadas con Jest y Supertest que verifique el correcto funcionamiento de cada endpoint, incluyendo casos de error.

- Desarrollar un frontend visual que consuma la propia API y permita al usuario interactuar con su biblioteca desde el navegador.
- Publicar el código en un repositorio público de GitHub y desplegar la aplicación en Vercel con URL pública accesible al evaluador.

3. Justificación

Las APIs REST son el estándar de comunicación entre sistemas en el desarrollo de software moderno. Páginas web, aplicaciones móviles, microservicios e integraciones entre plataformas se comunican mediante peticiones HTTP a APIs que exponen recursos. Por esta razón, el dominio del diseño y la construcción de APIs REST es una competencia fundamental en la formación de un desarrollador de backend.

Este proyecto fue elegido como vehículo de aprendizaje porque combina múltiples conceptos de la asignatura en un escenario cotidiano y comprensible. La gestión de libros ejercita las operaciones CRUD básicas, la validación de datos, la persistencia con SQLite y la paginación. Las estadísticas exigen consultas SQL agregadas y retorno de estructuras de datos compuestas. El buscador tolerante a acentos introduce el problema de la normalización Unicode en el contexto de bases de datos.

La elección de Node.js con Express sobre las otras tecnologías disponibles obedece a su ecosistema maduro de middlewares (`helmet` , `express-rate-limit` , `joi` , `pino`) que permiten construir una API con características de producción sin añadir complejidad arquitectónica innecesaria. La adición de Joi para la validación de esquemas, el patrón de capas (rutas → esquemas → servicios → base de datos) y la suite de tests con Jest y Supertest demuestran la aplicación práctica de los principios de calidad de código vistos en la asignatura.

El frontend integrado añade valor pedagógico adicional: demuestra cómo una API REST puede ser consumida por una interfaz de usuario real, cerrando el ciclo completo de una aplicación web. El uso de CSS personalizado con variables, animaciones y diseño responsivo sin frameworks externos evidencia dominio de las tecnologías base de la web.

4. Alcance del proyecto

4.1 Requerimientos funcionales

Código	Requerimiento	Endpoint
RF-01	Crear un nuevo libro con sus atributos validados	POST /libros
RF-02	Listar todos los libros con filtros opcionales por estado y género, y paginación	GET /libros
RF-03	Buscar libros por título o autor de forma tolerante a acentos	GET /libros/buscar? q=
RF-04	Consultar un libro específico mediante su identificador	GET /libros/:id
RF-05	Actualizar todos los campos de un libro existente	PUT /libros/:id
RF-06	Cambiar únicamente el estado de lectura de un libro	PATCH /libros/:id/estado
RF-07	Eliminar un libro del catálogo	DELETE /libros/:id
RF-08	Obtener estadísticas agregadas del catálogo (total, por estado, por género, calificación promedio, mejores calificados)	GET /stats
RF-09	Validar que el estado sea uno de los valores permitidos: pendiente, leyendo, leído	(validación)

RF-10	Validar que la calificación sea un entero entre 1 y 5	(validación)
RF-11	Retornar errores HTTP estándar con mensaje descriptivo cuando los inputs sean inválidos o el recurso no exista	(transversal)

4.2 Requerimientos no funcionales

Código	Categoría	Requerimiento
RNF-01	Seguridad	La API debe incluir headers de seguridad HTTP configurados con Helmet
RNF-02	Disponibilidad	La API debe estar desplegada públicamente con URL estable
RNF-03	Documentación	La API debe exponer documentación interactiva Swagger UI
RNF-04	Mantenibilidad	El código debe estar organizado en capas: rutas, esquemas, servicios y base de datos
RNF-05	Calidad	Toda funcionalidad debe estar cubierta por pruebas automatizadas con Jest
RNF-06	Portabilidad	La aplicación debe funcionar en cualquier entorno con Node.js 18 o superior
RNF-07	Rendimiento	Los endpoints deben responder en menos de 200 ms en condiciones normales
RNF-08	Versionado	El código debe estar versionado en un repositorio público de Git accesible al docente
RNF-09	Validación	Toda entrada del usuario debe ser validada con Joi antes de llegar a la lógica de negocio
RNF-10	Control de uso	La API debe aplicar rate limiting para limitar el abuso (máx. 100 req/15 min por IP)

4.3 Fuera del alcance

Las siguientes funcionalidades quedan explícitamente fuera del alcance de esta entrega:

- Autenticación y autorización de usuarios (no requerido para la actividad).
- Sistema de recomendaciones de libros basado en historial de lectura.
- Integración con APIs externas de catálogos de libros (Google Books, OpenLibrary).
- Exportación del catálogo en formatos CSV o Excel.
- Notificaciones o recordatorios de lectura.

5. Herramientas

5.1 Lenguaje y framework

Herramienta	Versión	Función
Node.js	18+	Runtime de JavaScript del lado del servidor
Express	5.x	Framework web para construir la API REST
Joi	18.x	Validación declarativa de schemas de entrada
Helmet	8.x	Middlewares de seguridad HTTP (14 headers)

express-rate-limit	8.x	Control de tasa de peticiones por IP
--------------------	-----	--------------------------------------

5.2 Persistencia y datos

Herramienta	Versión	Función
SQLite	embebido	Motor de base de datos relacional en un solo archivo
better-sqlite3	12.x	Driver síncrono de SQLite optimizado para Node.js

5.3 Logging y observabilidad

Herramienta	Versión	Función
Pino	10.x	Logging estructurado en formato JSON
pino-pretty	13.x	Formato legible con colores para entorno de desarrollo

5.4 Documentación

Herramienta	Versión	Función
swagger-ui-express	5.x	Interfaz interactiva para explorar y probar la API
yamljs	0.3.x	Carga del archivo OpenAPI 3.0 en formato YAML

5.5 Pruebas y calidad

Herramienta	Versión	Función
Jest	30.x	Framework de pruebas automatizadas
Supertest	7.x	Cliente HTTP para tests de integración de Express

5.6 Entorno y versionado

Herramienta	Función
Git	Sistema de control de versiones
GitHub	Hospedaje del repositorio remoto público
Vercel	Plataforma de despliegue en la nube (serverless)

6. Repositorio de código

El código completo del proyecto se encuentra disponible públicamente en el siguiente repositorio de GitHub:

<https://github.com/isaias-sanchez/api-biblioteca-personal>

El proyecto está desplegado y accesible en producción en:

<https://api-biblioteca-personal-six.vercel.app>

El repositorio contiene los siguientes elementos:

Elemento	Descripción
----------	-------------

src/app.js	Configuración de Express con middlewares de seguridad, logging y rutas
src/database.js	Conexión SQLite con creación automática del schema
src/schemas.js	Esquemas Joi para validación de todos los inputs
src/services.js	Capa de lógica de negocio separada de los controladores
src/logger.js	Logger Pino con middleware HTTP
src/routes/libros.js	Controladores del recurso libros (7 endpoints)
src/routes/stats.js	Controlador de estadísticas
src/__tests__/app.test.js	22 pruebas automatizadas con Jest y Supertest
public/index.html	Frontend de la biblioteca (HTML5)
public/style.css	Estilos del frontend (sidebar, panel deslizante, animaciones)
public/app.js	Lógica del frontend (consumo de la API, interactividad)
swagger.yaml	Especificación OpenAPI 3.0 completa
index.js	Punto de entrada del servidor
seed.js	Datos de ejemplo (9 libros clásicos)
vercel.json	Configuración de despliegue en Vercel
README.md	Instrucciones de instalación y uso

Para obtener el código, basta con clonar el repositorio:

```
git clone https://github.com/isaias-sanchez/api-biblioteca-personal.git
cd api-biblioteca-personal
npm install
npm start
```